KAVACH
Verification Agent

Current implementation status, pending work, integration flow and JSON contract

Repository review | 23 October 2026 | Branch: verification-agent

1. Scope and ownership

The Verification Agent evaluates a submitted cleaning completion using stored before/after evidence, checklist claims, task requirements and available location evidence. It returns a versioned JSON finding for the orchestrator/supervisor to route. It is not the worker task app, dispatch allocator, inspector queue, or incident-closure service.

This report describes the checked-out project folder as found. The three requested product features below are intentionally recorded as PENDING; source code for those features is not present in the current repository snapshot.

Item	Observed status	Practical meaning
Verification API	Implemented prototype	FastAPI endpoint validates a submission and returns structured findings.
Visual analysis	Partial prototype	Gemini or a mock analyzer describes visible observations; no semantic cleanup score/confidence yet.
Image quality	Implemented prototype	OpenCV checks blur, brightness, dimensions and near-duplicate images; it does not align camera viewpoints.
Worker-facing PWA	Pending	No browser app directory or camera-only capture flow exists in this folder.
Inspector routing	Pending integration	Response has a review boolean for current inconclusive/manual cases, but no explicit <80% confidence rule or inspector assignment/alert service.

Important boundary

The reference PDF supplied for this task specifies Field Worker execution and its status/JSON contract. It is used here as a format and integration reference. Its Module 06 implementation instructions do not change the Verification Agent's ownership. Verification consumes worker submissions; it must not generate dispatch decisions or claim work is verified/closed unless a separately approved downstream workflow assigns that action.

2. What the current project does

Current API route: **POST /api/v1/tasks/{task_id}/verification**. It expects an API key in **X-KAVACH-API-Key** and a request containing either a submission ID or a Field Worker completion contract. The service retrieves task and evidence records, checks their task ownership and required before/after references, then computes visual, checklist, GPS and score findings.

Pipeline step	Current behavior in code
Request and authentication	Pydantic request validation; development API-key dependency. Missing configured key returns 503; wrong key returns 401.
Task/evidence loading	Repository interfaces with demo memory repositories and SQLite-backed task/evidence/audit repositories. Evidence image paths are constrained under a trusted storage root.
Field Worker mapping	Adapter maps worker evidence types/statuses and completion location/checklist fields to verifier internal fields, preserving unknown/missing values.
Image file validation	JPEG/PNG/WebP decode and MIME, size and ownership checks before vision use.
Image quality	OpenCV sharpness, brightness, dimensions and dHash near-duplicate measurements; quality flags append limitations/reason codes.
Vision analysis	Mock fixture analyzer or Gemini structured JSON analyzer produces BEFORE/AFTER observations, visual change and limitations. Gemini does not choose PASS/FAIL.
Plan/GPS/decision	Plan compliance compares claims with observations; GPS checks configured location; weighted score and deterministic decision engine generate status and remarks.
Audit and response	Verification record is persisted/idempotent through repository implementation and returned with contract version, findings, remarks and evidence references.

Current response statuses

PASS means current deterministic rules support completion; **FAIL** means a supported failure condition; **INCONCLUSIVE** means evidence is missing or analysis cannot resolve the case; **MANUAL_REVIEW** is used for contradictory or limited evidence. A separate human reviewer assignment mechanism is not present.

Project caveats

The current configuration defaults to mock vision and memory repositories. A SQLite option exists. This is not a production PostgreSQL-only deployment. Authentication is an API-key development stub, not user/worker identity plus role-based access. These details must be agreed with the shared backend and supervisor team.

3. Pending work (keep all three requested items open)

Status below is based on the actual folder contents. Existing OpenCV image-quality checks and Gemini observation extraction are useful foundations, but they do not satisfy the requested landmark alignment, cleanup score, PWA, or confidence threshold behavior.

Priori	Pending item	Gap in current folder	Completion evidence
P0	Offline-first, lightweight Worker PWA; camera-only capture and client-side compression.	No PWA/frontend exists. No getUserMedia camera flow, offline capture queue, install manifest/service worker, compression UI, or worker API upload client.	Installable on target phone; camera permission capture only; no gallery/file picker; compressed image metadata; offline queue survives reload and syncs safely when online.
P0	Two-tier verification: landmark keypoint alignment + vision semantic cleanup scoring.	OpenCV currently measures quality and duplicate hashes only; it does not keypoint-match/register before and after images. Gemini returns observations/change/limitations, not a numeric cleanup score.	Tests for known aligned and mismatched views; alignment metrics and aligned frame passed to vision; JSON includes bounded cleanup_score; residual conditions remain visible.
P0	Human-in-the-loop: confidence below 80% flags manual review.	No verification_confidence field, threshold setting, low-confidence reason code, or explicit <80 rule. Current MANUAL_REVIEW for limitations is not this requirement.	At confidence 79.9: MANUAL_REVIEW, requires_manual_review=true and explicit reason code. At exactly 80: no low-confidence trigger. No auto-close on low confidence.
P1	Inspector queue / notification integration.	No inspector user identity, queue, notification, assignment API or callback.	Supervisor/orchestrator consumes review flag and records assignment/acknowledgement; reviewer decision returns through an agreed contract.
P1	Production runtime and shared contract decisions.	Auth stub and SQLite/memory alternatives; no shared identity provider, authoritative deployed DB/storage config, or frozen cross-agent payload version.	Agreed contract version, production auth/RBAC, shared DB/evidence store, integration tests against the real Field Worker endpoint.
P1	Real-photo and provider evaluation.	Tests mostly use fixtures/fake Gemini; no labeled dataset, calibrated confidence or operational review metrics.	Consent-safe labeled photo set, rubric, threshold calibration, false-pass/false-review analysis and monitored provider availability.

Do not mark the first three as complete until their acceptance evidence exists. In particular, current OpenCV image quality is not landmark keypoint alignment, and current Gemini descriptions are not a numeric cleanup score.

4. How the agents should interact

Expected chain, subject to the team's orchestrator contract:

```
Incident Intelligence -> Duplicate Detection -> Supervisor -> Sanitation -> Dispatch -> Field Worker -> Verification -> Supervi
```

The upstream agents provide an incident/task package and policy. Dispatch owns worker assignment. Field Worker owns task execution and capture. Verification receives a submission and reads server-stored evidence bytes by trusted evidence IDs. Verification returns findings; supervisor/orchestrator owns the review queue and final operational routing.

Producer -> consumer	Required handoff data	Boundary to confirm
Dispatch -> Field Worker	Accepted assignment/task IDs, worker identity, approved task/checklist, site policy, evidence requirements.	Only accepted assignments become executable; exact task/status identifiers must be frozen.
Field Worker -> Verification	Task/submission IDs, completed checklist step IDs, before/after evidence IDs, completion location event ID, secure stored-image references.	Evidence is accepted only when server storage confirms it; verifier rejects cross-task IDs.
Verification -> Supervisor/orchestrator	Contract version, status, score and score role, confidence, reason codes, failed requirement IDs, remarks and evidence references.	Supervisor decides handling for PASS/FAIL/INCONCLUSIVE/MANUAL_REVIEW; no silent close on low confidence.
Inspector -> Supervisor/orchestrator	Human review outcome, reviewer identity, rationale and timestamp.	Requires a new agreed API/queue contract; not implemented here.

Recommended delivery sequence

1) Agree the shared worker submission/evidence contract and inspector routing contract. 2) Build and secure the camera PWA against the agreed Field Worker API. 3) Add deterministic landmark registration and expose its diagnostics. 4) Extend the vision schema with cleanup_score and confidence. 5) Combine signals and enforce the 80% boundary centrally. 6) Add API/contract tests, real-device offline tests, image-pair tests, and supervisor queue integration. 7) Calibrate with labeled field photos before operational use.

The report's confidence requirement is a product rule: values below 80% route to human review, not rejection or automatic closure. It does not imply that the model's confidence is statistically reliable; calibration and a reviewer route are separate prerequisites.

5. Current API and JSON contract

Request example

```
POST /api/v1/tasks/TASK-100/verification
X-KAVACH-API-Key: <configured-development-key>
Content-Type: application/json

{
  "contract_version": "1.1",
  "submission_id": "SUB-123"
}
```

A Field Worker completion payload may be supplied in place of a previously stored submission ID. The request contract has task/submission references, checklist_completed, before_evidence_ids, after_evidence_ids and completion_location_event_id. Image bytes are not uploaded to this verifier endpoint; it resolves bytes from the trusted server-side evidence repository.

Current response shape (trimmed)

```
{
  "contract_version": "1.1",
  "verification_id": "VER-...",
  "task_id": "TASK-100",
  "submission_id": "SUB-123",
  "verification_status": "PASS | FAIL | INCONCLUSIVE | MANUAL_REVIEW",
  "verification_score": 0.0,
  "score_status": "AVAILABLE | PARTIAL | UNAVAILABLE",
  "score_breakdown": {"image_evidence": 0.0, "gps_consistency": 0.0, "plan_compliance": 0.0},
  "image_findings": {
    "before_observations": {"overflow": "PRESENT", "scattered_waste": "PRESENT", "path_obstructed": "PRESENT"},
    "after_observations": {"overflow": "ABSENT", "scattered_waste": "ABSENT", "path_obstructed": "ABSENT"},
    "visual_change": "IMPROVED",
    "limitations": []
  },
  "location_finding": {"status": "CONSISTENT", "score": 100.0, "reasons": []},
  "plan_compliance_findings": [],
  "remarks": [],
  "requires_manual_review": false,
  "reason_codes": [],
  "evidence_refs": ["EVIDENCE-BEFORE", "EVIDENCE-AFTER"],
  "created_at": "<ISO-8601>"
}
```

Fields required for the pending confidence feature

Additive response field	Expected meaning
verification_confidence	0-100 combined prototype estimate; derive conservatively from alignment and semantic confidence. Null/unavailable must never be converted to a high value.
alignment_finding	Registration status plus match/inlier diagnostics, confidence and failure reason; never include raw image bytes.
image_findings.cleanup_score	0-100 semantic visual cleanup estimate based on conditions before and after; keep separate from final PASS/FAIL.
image_findings.confidence	Model self-assessment for visual observations and cleanup score; explicitly label as uncalibrated until validated.
requires_manual_review + reason_codes	True below 80 or when evidence is inconclusive; add LOW_VERIFICATION_CONFIDENCE and/or VERIFICATION_CONFIDENCE_UNAVAILABLE.

Maintain an additive contract update and coordinate the version with Field Worker and Supervisor owners before publishing. The attached Field Worker PDF uses illustrative schemas and explicitly calls for shared-contract agreement; do not copy its sample enums blindly into this verifier.

6. Verification folder map

The following table maps the current source tree to its responsibility. Virtual environments, pytest caches, temporary API test folders, and the local audit database are development/runtime artifacts and are not application source.

Path	Purpose
app/main.py	Creates FastAPI app, initializes repositories and vision adapter, registers route and health endpoints.
app/config.py; app/security.py	Environment-backed runtime settings; development API-key authentication dependency.
app/api/verification_routes.py	Protected POST verification endpoint and HTTP error mapping.
app/contracts/v1.py	Versioned inter-agent enums and request/response boundary contracts.
app/adapters/agent_contracts.py	Validates/maps Field Worker completion, evidence, GPS events and Sanitation plan fields.
app/schemas/	Pydantic request, response, GPS and visual observation schemas.
app/repositories/	Repository interfaces plus demo-memory and SQLite task/evidence/audit implementations.
app/services/verification_service.py	Coordinates evidence checks, vision, GPS, plan compliance, scoring, decisions, remarks and audit response.
app/services/vision_model_service.py	Mock and Gemini vision analyzers, image loading/validation, structured-output parsing and bounded retries.
app/services/image_quality_service.py	OpenCV sharpness, brightness, dimensions and perceptual duplicate audit.
app/services/location_service.py; gps_verification_service.py	Distance, accuracy, coordinate, timestamp and location-source checks.
app/services/decision_engine.py; score_calculator.py; plan_compliance_service.py; remarks_service.py	Deterministic outcome rules, weighted/partial score, requirement findings and human-readable remarks.
app/services/image_evidence_service.py; app/validators/	Evidence collection/completeness, task ownership, trusted bytes and format validation.
tests/	Six focused test modules covering API, contracts, Gemini adapter, image quality, scenario outcomes and requested fixes.
scripts/try_gemini.py; check_image_quality.py	Manual Gemini/provider diagnostic and local image-quality diagnostic scripts.
docs/INTEGRATION.md	Agent boundary, source-field mappings, API examples and status behavior.
docs/REQUIREMENTS_COVERAGE.md	Mapping from the Verification guide requirements to implementation and tests; includes known prototype limits.
docs/specs/	Seven PDF references for other KAVACH agents; the Verification technical guide referenced by coverage is not stored in this folder.
requirements.txt; .env.example; pytest.ini; README.md	Dependency declarations, sample settings, pytest configuration and minimal project entry point.

7. Test status, risks and acceptance checklist

Test inventory in the folder

Test file	Coverage area
tests/test_verification_api.py	HTTP auth, request validation, response contract, lifespan/health and image-quality route cases.
tests/test_agent_contracts.py	Field Worker/Sanitation mapping, shared contract schemas, evidence/location ownership and repositories.
tests/test_gemini_vision_adapter.py	Structured Gemini parsing, retries/errors, schema rejection, privacy, stored-image validation.
tests/test_image_quality_service.py	Blur, brightness, resolution, duplicate hashes, missing bytes and audit metadata.
tests/test_pdf_scenarios.py	End-to-end deterministic outcomes, scores, contradictions and idempotency.
tests/test_requested_fixes.py	GPS/range/policy and requested regression fixes.

This report reviewed the tests and source coverage map but did not complete a fresh full test run. Existing pass counts in the coverage document should be re-run in the intended project virtual environment before merge.

Acceptance checklist for the pending suggestions

Check	Pass condition
Camera-only	App has no gallery/file picker route; browser camera permission is required; task/type are visible before capture.
Offline PWA	App shell opens offline; compressed captured photos persist with task/type/operation metadata; retries are safe and no capture is silently discarded.
Landmark alignment	Known same-scene images align to a shared frame and show match/inlier diagnostics; unrelated scenes fail conservatively and request review.
Semantic cleanup scoring	Vision response validates observations, cleanup_score 0-100, confidence 0-100 and limitations; schema rejects unsupported/extra decision fields.
80% rule	79.9 routes MANUAL_REVIEW and sets the review flag; exactly 80 is not flagged by this threshold alone; missing confidence cannot pass silently.
Human route	Supervisor/inspector integration records the reviewer, decision, rationale and time; low confidence does not auto-reject or auto-close.
Integration	Contract tests cover Field Worker upload/evidence IDs and Supervisor consumption of the verifier response.

Known risks

Model confidence may be overconfident and must be calibrated on representative labeled photos. Keypoint alignment can fail under viewpoint, lighting or scene changes and is not proof of authenticity. Offline browser storage can be cleared or evicted. Real Gemini calls can fail for credentials, model access, quotas or network reasons. The current inspector flag has no destination until Supervisor/orchestrator integration is defined.

Source basis

Repository files reviewed: docs/INTEGRATION.md, docs/REQUIREMENTS_COVERAGE.md, current app API/contracts/schemas/services/configuration, requirements.txt, and test inventory. Reference format and handoff-boundary context: KAVACH_New_Field_Worker_Execution_Task_Status_and_JSON_Contract.pdf. The reference itself distinguishes Module 06 ownership from Verification and describes its JSON examples as structural/illustrative pending team agreement.